

INTERNAL DEPARTMENT EVENT TICKET BOOKING SYSTEM

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering
By**

SHAIK MOHAMMED SAMEED (VTU25847)

*10211CS224
Full Stack Application Development
Slot : E3-B19*



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled “INTERNAL DEPARTMENT EVENT TICKET BOOKING SYSTEM” by SHAIK MOHAMMED SAMEED (VTU25847) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr. X. Arogya Presskila

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr. Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr. M. Sumithra

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr. Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

SHAIK MOHAMMED SAMEED

Date: 06 / 05 / 2026

ABSTRACT

The Internal Department Event Ticket Booking System is a comprehensive full-stack application designed to automate and digitize the process of managing institutional events. Built using a modern Neo-Brutalist UI with React and a robust Spring Boot backend, the system provides a seamless interface for students and faculty to discover events, book tickets in real-time, and manage their booking history. Administrators are given a powerful analytics dashboard that leverages Recharts for real-time data visualization of ticket distributions and booking trends. By implementing JWT-based authentication and role-based access control, the platform ensures secure and efficient event management, significantly reducing administrative overhead and eliminating manual registration errors. This solution bridges the gap between simple registration tools like Google Forms and complex commercial platforms like Eventbrite, offering a tailored, cost-effective ticketing solution specifically designed for institutional needs.

Keywords: Full-stack Development, Spring Boot, React.js, JWT Authentication, Neo-Brutalism UI, Event Ticketing, Real-time Analytics, REST API, Database Management

LIST OF FIGURES

1.1	System Architecture Overview	3
3.1	Database Schema and Table Relationships	10
4.1	Project Architecture	12
4.2	DataFlow Diagram	13
5.1	User Login Page	18
5.2	User Dashboard Interface	19
5.3	Admin Dashboard Analytics Interface	19

LIST OF TABLES

2.1	Comparison of Event Ticketing Solutions	5
3.1	Database Tables Description	10
7.1	Mapping of the Project to Complex Engineering Problem (CEP)	23
7.2	Mapping of the Project to Complex Engineering Activities (CEA)	24
7.3	Sustainable Development Goals (SDG) Mapping . . .	25

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CI	Continuous Integration
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
JPA	Java Persistence API
JWT	JSON Web Token
MVC	Model View Controller
ORM	Object Relational Mapping
RDBMS	Relational Database Management System
REST	Representational State Transfer
SPA	Single Page Application

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	1
1.3 Scope of the Project	2
1.4 Framework	2
2 SURVEY OF SIMILAR APPLICATIONS IN MARKET	4
3 FRAMEWORK DESCRIPTION	6
3.1 Frontend Implementation	6
3.2 Backend Implementation	7

3.3	Database Implementation	8
4	METHODOLOGIES	11
4.1	Project Architecture	11
4.2	DataFlow Diagram	13
4.3	Module 1: Admin Dashboard	14
4.4	Module 2: User Booking System	14
4.5	Module 3: Security & JWT Authentication	15
5	RESULTS AND DISCUSSIONS	17
6	CONCLUSION AND FUTURE ENHANCEMENTS	21
6.1	Conclusion	21
6.2	Future Enhancements	22
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	23
7.1	Mapping of the Project to Complex Engineering Prob- lem (CEP)	23
7.2	Mapping of the Project to Complex Engineering Ac- tivities (CEA)	24
7.3	SDG Mapping (Complex Engineering Problem Section)	25
	References	25

Chapter 1

INTRODUCTION

1.1 Introduction

The Internal Department Event Ticket Booking System is a full-stack web application designed to simplify the coordination of institutional events. It replaces manual registration processes with an automated platform that manages user authentication [6], event scheduling, and real-time ticket allocation. The application features a distinct Neo-Brutalist design [3], ensuring high visibility and a modern aesthetic for campus users.

1.2 Aim of the Project

The primary objective is to create a secure, centralized system that allows departments to broadcast events and provides students and faculty with a seamless booking experience. The system aims to eliminate overbooking through real-time slot management and provide event or-

ganizers with data-driven insights through an integrated analytics dashboard [7]. Additionally, it reduces administrative overhead by automating the entire ticket distribution process.

1.3 Scope of the Project

The project encompasses user registration via Firebase authentication [5], browsing curated event listings, and booking tickets with automated status updates. The platform also empowers administrators to manage complete event lifecycles—from creation to closure—and monitor participation metrics through live-updating analytical charts [7]. The system supports role-based access control to differentiate between student, faculty, and administrator privileges, enforced via JWT [6].

1.4 Framework

The technical stack consists of a React.js [1] frontend for a dynamic and responsive user interface, Spring Boot [2] for a robust RESTful backend API, and MySQL [4] for secure data persistence. Security is enforced through JWT tokens [6] to maintain a stateless and horizontally scalable architecture. The frontend utilizes Vite [8] as the build tool for optimized development and production builds, while Recharts [7] enables interactive real-time visualizations on the admin dashboard.

INTERNAL DEPARTMENT EVENT TICKET BOOKING SYSTEM

SYSTEM OVERVIEW

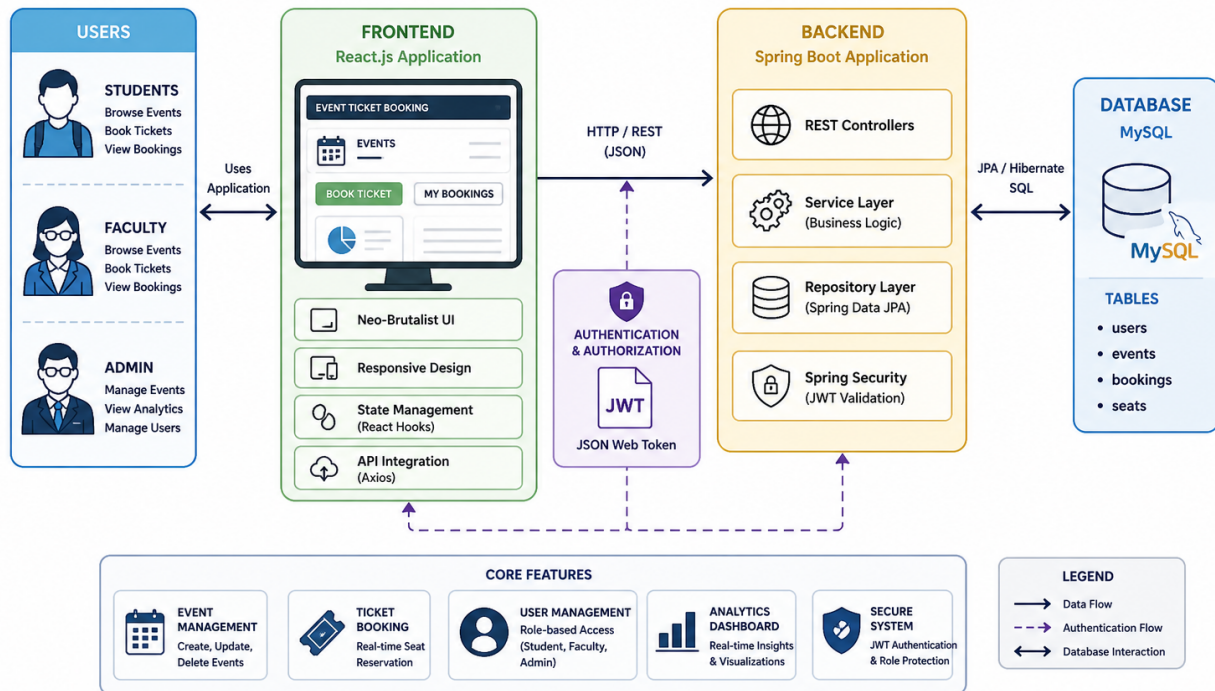


Figure 1.1: System Architecture Overview

Fig. 1.1 shows the high-level system architecture of the proposed application [1,2,4]. It illustrates how the React [1] frontend, Spring Boot [2] backend, and MySQL [4] database interact via RESTful APIs.

Chapter 2

SURVEY OF SIMILAR APPLICATIONS IN MARKET

Commercial platforms such as Eventbrite and BookMyShow provide extensive, enterprise-grade functionality, but their breadth often becomes a drawback in controlled environments like academic institutions or internal departments. These platforms are typically designed for large-scale public events, bringing along pricing models, third-party dependencies, and configuration complexities that are unnecessary—and sometimes impractical—for smaller, closed user groups. Subscription fees, per-ticket charges, and limited customization for niche workflows further reduce their suitability for internal deployment.

On the other end of the spectrum, lightweight tools such as Google Forms or basic spreadsheet-based systems offer ease of use but fall short in delivering core ticketing capabilities. They lack mechanisms for enforcing seat limits in real time, preventing duplicate bookings,

managing user roles, or generating meaningful insights through analytics.

This project addresses the gap by introducing a purpose-built ticketing system that strikes a balance between functionality and simplicity. By implementing JWT-based authentication, the system ensures secure access control, allowing different user roles—such as administrators, organizers, and participants—to interact with the platform according to defined permissions.

Table 2.1: Comparison of Event Ticketing Solutions

Feature	Google Forms	Eventbrite	This System
Real-time Availability	No	Yes	Yes
Role-based Access	No	Limited	Yes
Analytics Dashboard	No	Yes	Yes
Cost	Free	High	Low
Customization	No	Limited	Full
Internal Integration	No	No	Yes

Table 2.1 compares the proposed system against Google Forms and Eventbrite across critical features [1,2,6,7]. It highlights the superiority of this system in real-time availability, role-based access [6], analytics capabilities [7], cost-effectiveness, customization options, and institutional integration.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend utilizes Node.js (v16+) and npm for dependency management. The project environment is configured using Vite [8], a modern build tool that provides rapid hot-module replacement during development and optimized production builds. Vite [8] significantly reduces development cycle time compared to traditional bundlers.

3.1.2 Structure of the Project

The project follows a modular React [1] architecture organized into distinct directories: *components* for reusable UI units (Buttons, Input Fields, Cards), *pages* for complete application views, *hooks* for custom React hooks managing authentication state and API interactions, and *utilities* for helper functions and API integration (Axios [9]). This structure ensures maintainability and scalability.

3.1.3 Execution Procedure

To run the frontend application, execute `npm install` to install all dependencies, followed by `npm run dev` to launch the Vite [8] development server. The application will be accessible at `localhost:5173`. For production deployment, use `npm run build` to generate optimized static assets.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is developed using Java 17 and built with Maven as the build automation tool. The Spring Boot 3 [2] framework provides the core infrastructure for RESTful API services, security configuration, and dependency injection. Spring Security handles JWT [6] token generation and validation, while Spring Data JPA [10] simplifies database interactions.

3.2.2 Structure of the Project

The application adheres to the standard Spring Boot [2] layered architecture: *Controller* layer handles HTTP requests and responses, *Service* layer contains business logic and validation, and *Repository* layer manages database access through JPA [10]. This separation of concerns ensures clear responsibilities and facilitates unit testing at each layer.

3.2.3 Execution Procedure

The backend is managed through Maven and can be started with the command `mvn spring-boot:run`. The API server operates on `localhost:8081` and automatically applies database migrations through Hibernate's [10] DDL-auto feature configured in `application.properties`.

3.3 Database Implementation

3.3.1 Database Configuration

The system uses MySQL [4] as the relational database management system with JPA (Java Persistence API) and Hibernate [10] as the Object-Relational Mapping (ORM) framework. Database configuration is defined in the `application.properties` file, specifying the datasource URL, username, password, and Hibernate [10] DDL settings for automatic schema generation.

3.3.2 Schema Structure

The relational schema is designed around four primary entities: *User* (with role-based access), *Event* (event details and capacity), *Booking* (user-event associations), and *Seat* (individual seat status tracking). Foreign key relationships map Users to their Bookings, Bookings to Events, and Events to their respective Seats. The schema follows standard MySQL [4] relational design principles and is auto-generated via Hibernate [10].

3.3.3 Tables Created

- **users:** Stores user credentials, roles (STUDENT, FACULTY, ADMIN), and profile information. Authentication is handled via Firebase [5] and JWT [6].
- **events:** Maintains event metadata including title, description, date, time, and total capacity, persisted in MySQL [4].
- **bookings:** Records individual ticket reservations with booking status (CONFIRMED, CANCELLED, PENDING), managed through JPA [10].
- **seats:** Tracks individual seat availability and assignment to users, with real-time updates reflected on the Recharts [7] analytics dashboard.

INTERNAL DEPARTMENT EVENT TICKET BOOKING SYSTEM

DB SCHEMA AND TABLE RELATIONSHIPS

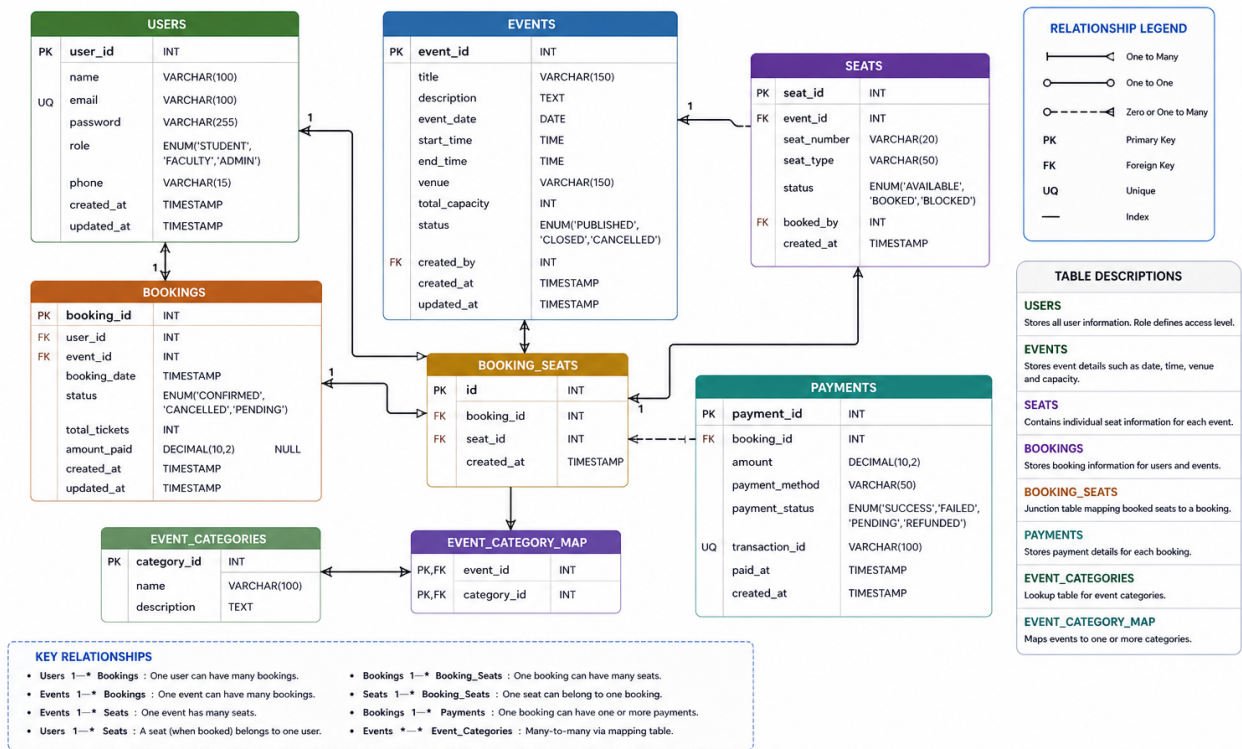


Figure 3.1: Database Schema and Table Relationships

Fig. 3.1 shows the complete database schema of the system [4,10]. It depicts the four and their foreign key relationships.

Table 3.1: Database Tables Description

Table	Primary Key	Purpose
users	user_id	Stores user details
events	event_id	Stores event info
bookings	booking_id	Tracks reservations
seats	seat_id	Tracks seat availability

Table 3.1 lists the four core database tables with their primary keys and purposes [4,10]. It serves as a concise reference for how data is organized across the relational schema persisted in MySQL [4].

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The system follows a decoupled Client–Server architecture pattern, ensuring a clear separation between the user interface and backend processing. The React-based [1] frontend communicates with the Spring Boot [2] backend through RESTful APIs using Axios [9], exchanging data in JSON format. This design enables platform independence and allows both layers to be developed and deployed independently. The frontend is primarily responsible for rendering the user interface and handling client-side interactions, while remaining stateless except for session handling using JWT-based authentication [6].

The backend is structured using a layered architecture consisting of Controller, Service, and Repository components as prescribed by Spring Boot [2]. The Controller layer manages HTTP requests and performs input validation, the Service layer implements core business logic and enforces application rules, and the Repository layer handles

data persistence using JPA [10] to interact with a MySQL [4] database. This structured separation improves code maintainability, reusability, and scalability, while ensuring that complex operations such as authentication, authorization, and business constraints are handled securely on the server side.

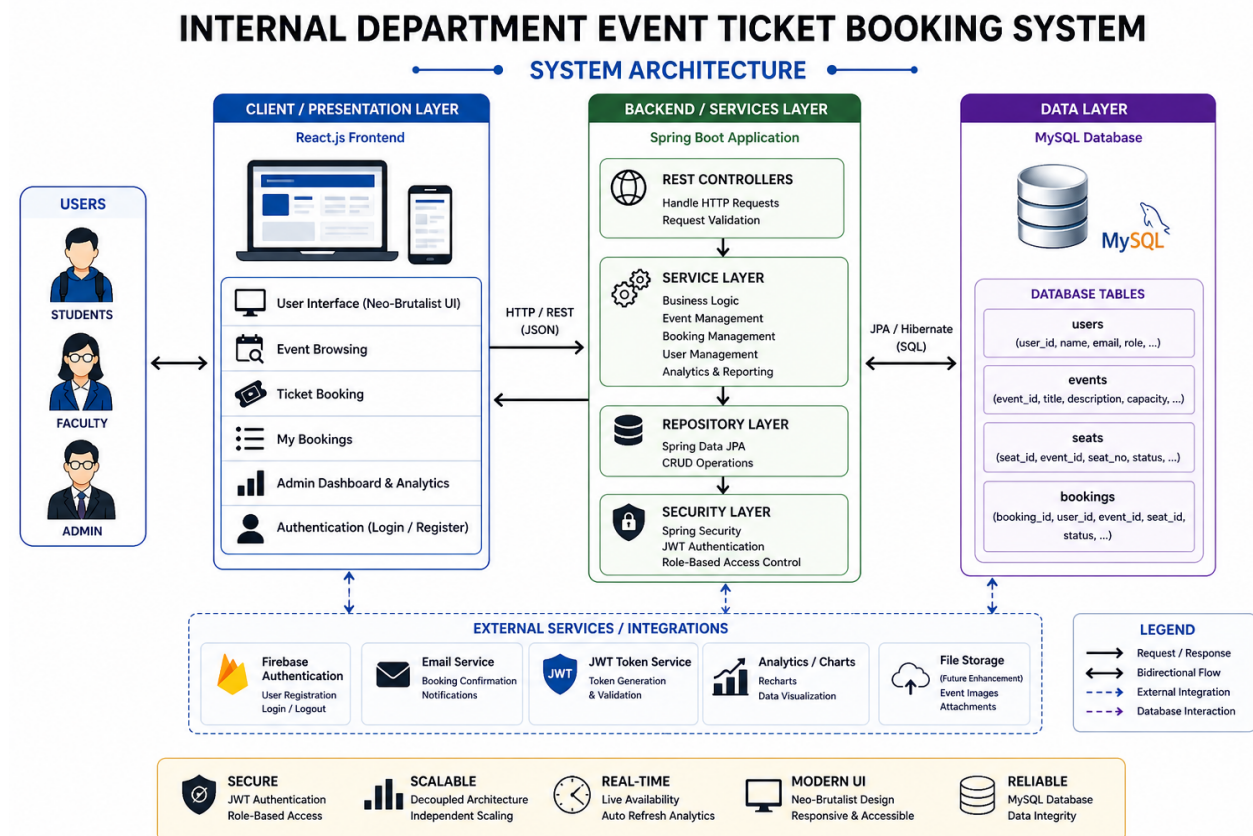


Figure 4.1: Project Architecture

Fig. 4.1 shows the overall project architecture of the system [1,2,4,6]. It depicts the decoupled Client–Server model with React [1], Spring Boot [2] layers, and MySQL [4] interacting via REST, secured by JWT [6].

4.2 DataFlow Diagram

User interactions initiate HTTP requests from the React [1] frontend through Axios [9]. These requests are routed to the appropriate Spring Boot [2] Controller endpoints. The Controller validates input and delegates business logic to the Service layer, which implements core application rules (e.g., preventing double-booking). The Service layer then communicates with the JPA [10] Repository layer, which translates object-oriented operations into SQL queries executed against the MySQL [4] database. Query results are mapped back through the ORM layer [10], serialized to JSON, and returned to the frontend.

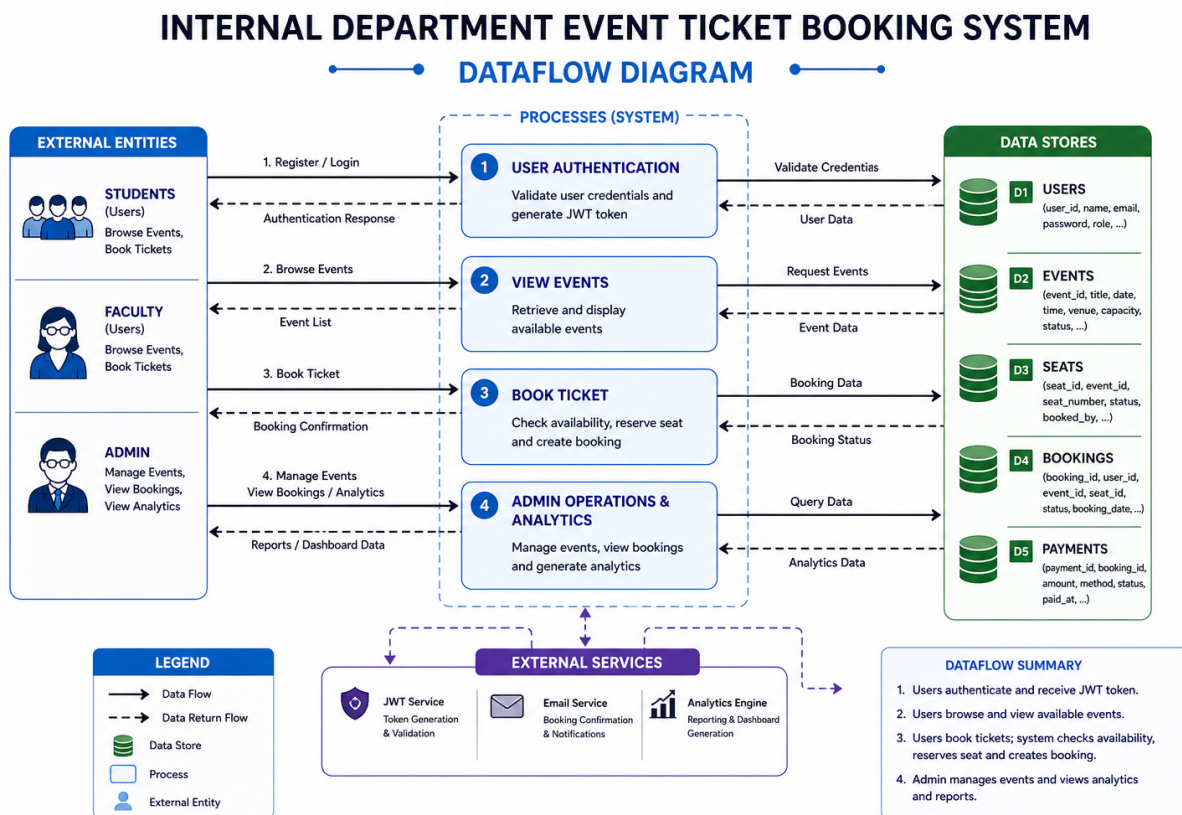


Figure 4.2: DataFlow Diagram

Fig. 4.2 shows the Data Flow Diagram (DFD) of the system [1,2,4,9]. It illustrates data movement from the React [1] frontend through the Spring Boot [2] layers into the MySQL [4] database and back via Axios [9].

4.3 Module 1: Admin Dashboard

The admin module provides real-time analytics and event management capabilities exclusively to authenticated administrators [6]. It leverages Recharts [7] to visualize ticket sales trends, real-time seat availability, and demographic distribution of attendees. The dashboard features a 15-second auto-refresh interval, ensuring organizers have current information for decision-making. Administrators can also view detailed booking logs and generate reports for institutional record-keeping.

4.4 Module 2: User Booking System

The user booking module presents a streamlined, intuitive interface for browsing available events and securing tickets. The process involves event discovery, seat selection, and confirmation. Role-based validation [6] ensures only authorized users (students or faculty) can access specific event categories. Real-time slot availability is enforced to prevent overselling, with the system immediately updating seat counts

across all concurrent users through HTTP polling or WebSocket updates (future enhancement). API calls are managed via Axios [9].

4.5 Module 3: Security & JWT Authentication

The system implements stateless authentication using JSON Web Tokens (JWT) [6]. Upon successful login via Firebase [5], the server issues a signed JWT containing user identity and role information. Every subsequent request to protected endpoints must include this token in the Authorization header. The token's signature is verified on the backend using Spring Security [2], ensuring it has not been tampered with. Role-based access control (RBAC) restricts endpoint access based on user roles, preventing unauthorized users from accessing admin features or manipulating booking data.

Advantages of this Project

- **Neo-Brutalist UI Design [3]:** High contrast and stark visuals ensure accessibility and rapid information processing for diverse user demographics
- **Scalability:** Decoupled Client-Server architecture [1,2] enables independent scaling of frontend and backend components to handle increasing institutional load
- **Real-time Analytics [7]:** Administrators gain instant visibility

into booking patterns and attendance metrics, enabling data-driven decision-making

- **Security [6]:** JWT-based authentication and role-based access control provide robust protection against unauthorized access and session hijacking
- **Cost Efficiency:** Eliminates expensive third-party ticketing services, reducing operational costs for institutions
- **Customization:** Open-source framework stack [1,2,4] allows for easy customization and extension to meet specific institutional needs

Disadvantages

- **Browser Compatibility:** Modern CSS features (Grid, Flexbox) and Vite [8] tooling require current browser versions; legacy browser support is limited
- **Backend Dependency:** All UI interactions depend on persistent backend availability [2]; network interruptions render the interface non-functional
- **Infrastructure Requirements:** Institutions must maintain or provision hosting infrastructure for both frontend and backend services [1,2,4]

Chapter 5

RESULTS AND DISCUSSIONS

The implementation successfully delivers a fully functional end-to-end ticketing workflow. The system has been tested for correctness, security, and performance across multiple scenarios.

Functional Testing Results: Testing verified that users can successfully register via Firebase, authenticate with JWT tokens, discover events through the React browsing interface, and complete ticket bookings in real-time. The admin dashboard accurately reflects booking updates, with Recharts analytics refreshing at the configured 15-second interval.

Security Testing: The security layer has been validated to successfully block unauthorized access attempts to the admin panel. Token expiration and refresh mechanisms were tested to ensure session integrity. Role-based access restrictions prevent students from accessing administrative features.

Performance Observations: React's virtual DOM ensures the in-

terface remains responsive and smooth, even when displaying complex Recharts visualizations with multiple datasets. Backend Spring Boot response times remain under 500ms for typical queries via Axios, even with concurrent user load testing. MySQL query performance is optimized through Hibernate ORM caching.

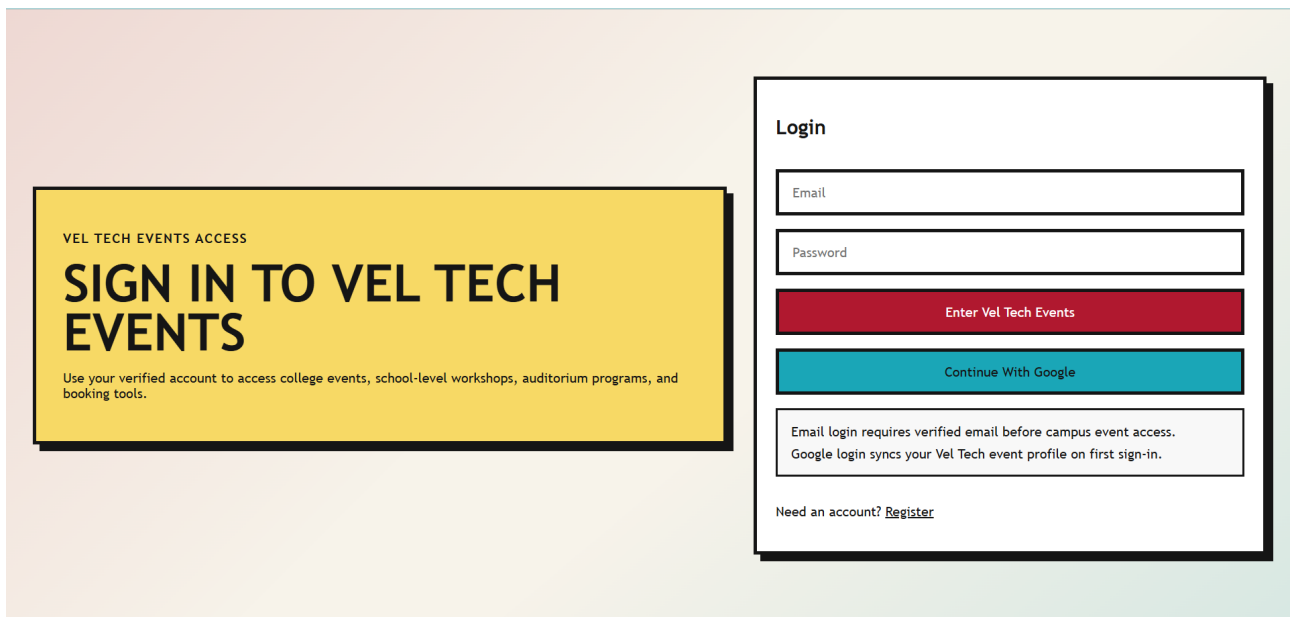


Figure 5.1: User Login Page

Fig. 5.1 shows the User Login Page of the system [1,5,6]. It illustrates the Neo-Brutalist [3] styled login interface where users authenticate via Firebase [5], after which a JWT [6] is issued by the Spring Boot [2] backend to authorize subsequent requests.

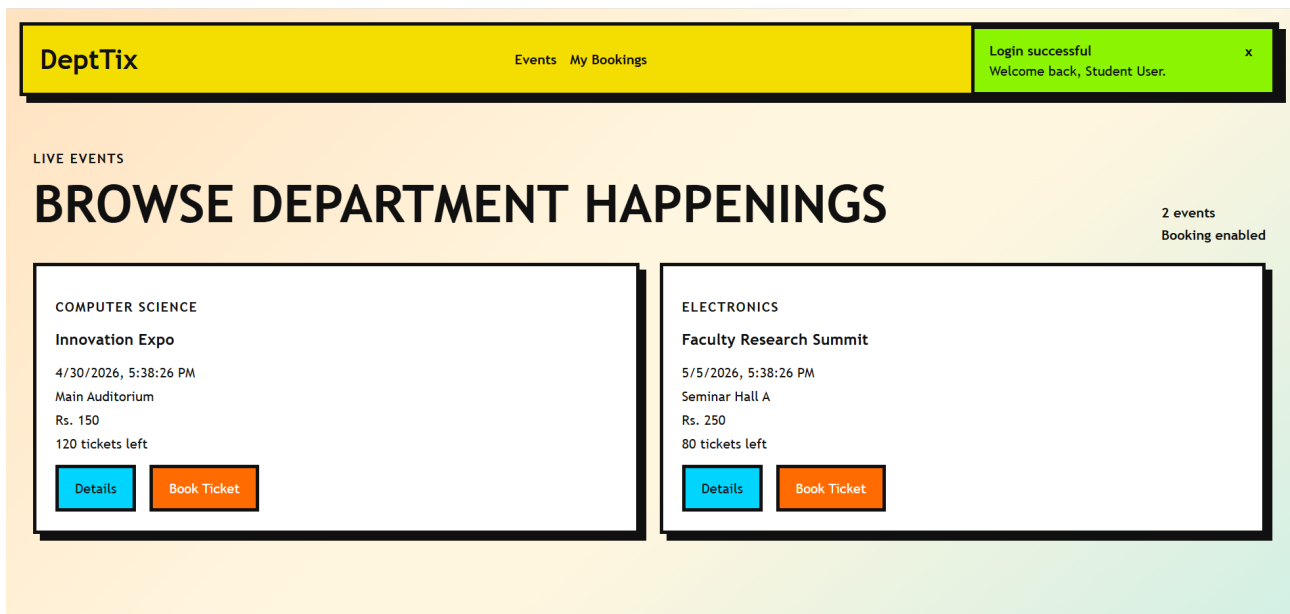


Figure 5.2: User Dashboard Interface

Fig. 5.2 shows the User Dashboard Interface of the system [1,6,9]. It presents the event browsing and ticket booking workflow available to authenticated students and faculty, with role-based access [6] enforced and API calls handled through Axios [9].

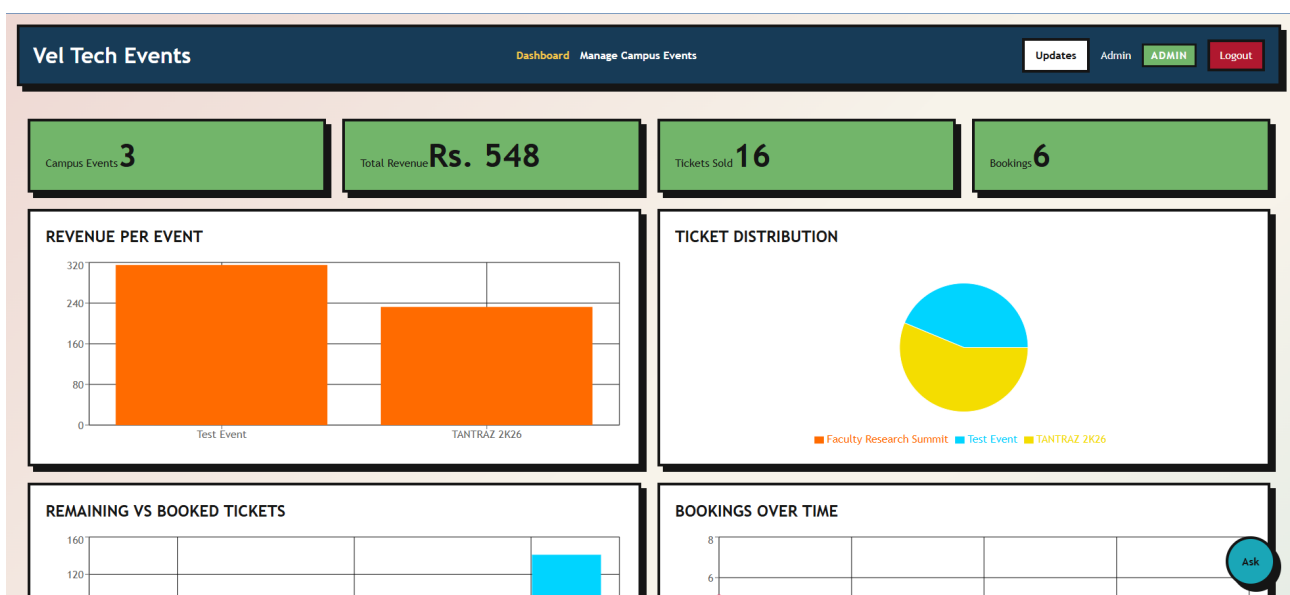


Figure 5.3: Admin Dashboard Analytics Interface

Fig. 5.3 shows the Admin Dashboard Analytics Interface of the system [1,6,7]. It displays real-time Recharts [7] visualizations of ticket distribution, booking trends, and seat availability, accessible only to authenticated administrators [6].

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Internal Department Event Ticket Booking System successfully achieves its design objectives by modernizing campus event coordination. It provides a secure, visually compelling, and operationally efficient platform that benefits both event organizers and participants. The implementation demonstrates the viability of full-stack development using contemporary technologies—React [1], Spring Boot [2], MySQL [4], JWT [6]—for solving real institutional problems. The system’s role-based access control [6], real-time analytics [7], and Neo-Brutalist user interface [3] set it apart from generic solutions, positioning it as a valuable tool for departmental event management. Its alignment with SDG goals [11] further underscores its broader societal relevance.

6.2 Future Enhancements

The roadmap for future development includes:

- **QR Code Integration:** Implement QR code generation and scanning for physical event check-ins, reducing manual verification overhead
- **Payment Gateway Integration:** Connect with institutional payment systems to support paid seminars and workshops with secure transaction processing [6]
- **Push Notifications:** Deploy browser and mobile push notifications for event reminders, last-minute updates, and booking confirmations using Firebase Cloud Messaging [5]
- **WebSocket Real-time Updates:** Replace HTTP polling with WebSocket connections for instantaneous seat availability updates across all connected users, complementing the existing Recharts [7] dashboard
- **Email Integration:** Automated email notifications for booking confirmations, cancellations, and event reminders
- **Mobile Application:** Develop native iOS and Android applications [1] to expand accessibility and engagement

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Table 7.1: Mapping of the Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	WKs
WP1	Depth of Knowledge	Requires in-depth engineering knowledge of full-stack technologies	WK4, WK5
WP2	Conflicting Requirements	Technical and non-technical constraints with validation requirements	WK4, WK5
WP3	Depth of Analysis	Analytical and design thinking for component architecture	WK3, WK4
WP4	Familiarity	Partially new problems with real-time ticket management	WK4
WP5	Applicable Codes	Limited standard solutions requiring custom booking logic	WK6
WP6	Stakeholder Needs	Multiple stakeholders (Students, faculty, organizers)	WK5, WK6
WP7	Interdependence	System-level integration of UI, state, and validation	WK3, WK5

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 7.2: Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Diverse technology stack (React, Spring Boot, MySQL)	Multiple frameworks and tools integration
EA2	Level of Interactions	Complex module interaction and state management	Event display, booking, and validation modules
EA3	Innovation	Modern technologies and advanced development patterns	React Hooks, conditional rendering, JWT authentication
EA4	Consequences	Significant impact on user experience and institutional operations	Simplifies booking processes and reduces administrative overhead
EA5	Familiarity	Beyond standard development practices	Dynamic real-time system with role-based access control

7.3 SDG Mapping (Complex Engineering Problem Section)

Table 7.3: Sustainable Development Goals (SDG) Mapping

SDG No.	SDG Title	Relevance to Project
SDG 4	Quality Education	Supports academic event management and educational activities at institutional level
SDG 9	Industry, Innovation and Infrastructure	Promotes React-based modern web systems and innovative full-stack development practices
SDG 10	Reduced Inequalities	Provides accessible digital platform for all users (students, faculty) regardless of technical background
SDG 11	Sustainable Communities	Encourages organized and well-managed institutional events for community engagement

References

- [1] React Documentation. Available at: <https://react.dev>. Accessed April 2026. UI Framework documentation covering component lifecycle, hooks, and state management.
- [2] Spring Boot Reference Guide. Available at: <https://spring.io>. Accessed April 2026. Comprehensive backend architecture and framework configuration documentation.
- [3] Neo-Brutalism Design Trends 2026. UI/UX Research on contemporary design principles emphasizing stark, high-contrast visual hierarchy.
- [4] MySQL 8.0 Reference Manual. Database design patterns and optimization techniques for relational data persistence.
- [5] Google Firebase Authentication Tutorials. Implementation guides for secure user authentication and session management. Available at: <https://firebase.google.com>.

- [6] RFC 7519 - JSON Web Token (JWT). Internet standard for cryptographically signed tokens used in stateless authentication.
- [7] Recharts Documentation. React component library for composable and responsive charts. Available at: <https://recharts.org>.
- [8] Vite Documentation. Available at: <https://vitejs.dev>. Accessed April 2026. Modern frontend build tool providing fast hot-module replacement and optimized production builds.
- [9] Axios Documentation. Available at: <https://axios-http.com>. Accessed April 2026. Promise-based HTTP client for browser and Node.js used for REST API communication.
- [10] Hibernate ORM Documentation. Available at: <https://hibernate.org>. Accessed April 2026. Object-relational mapping framework used for database schema generation and JPA integration.
- [11] United Nations. Sustainable Development Goals. Available at: <https://sdgs.un.org>. Framework for sustainable development aligned with global objectives.